



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA



Projeto de SD

Googol: Motor de pesquisa de páginas Web

Henrique Diz - 2023213681

Rodrigo Manhão - 2023207589

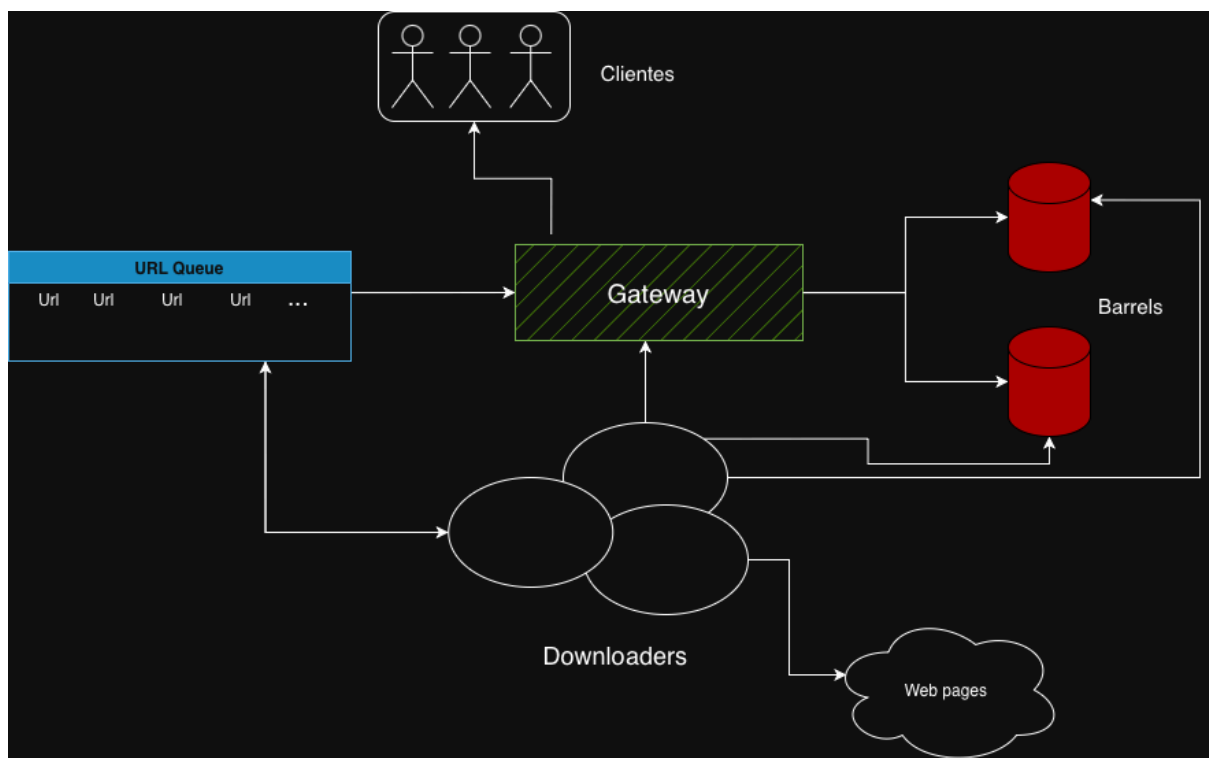
João Francisco - 2023228417

1. Introdução

O presente projeto tem como objetivo o desenvolvimento de um sistema distribuído de indexação e pesquisa de informação na web, inspirado no funcionamento básico de motores de busca como o Google. O sistema foi concebido com base em princípios de Sistemas Distribuídos, utilizando Java RMI (Remote Method Invocation) como mecanismo principal de comunicação remota entre processos.

2. Arquitetura do Projeto

A arquitetura do sistema segue um modelo distribuído modular, onde cada componente desempenha um papel específico e comunica com os demais através de invocações remotas de métodos (RMI). A arquitetura do projeto foi desenvolvida de forma a garantir escalabilidade e tolerância de falhas entre os diferentes processos.



(Fig 1 - Arquitetura do projeto)

2.1 Visão geral do Sistema

O sistema é composto por cinco módulos principais:

1. **Client** - Interface de interação com o utilizador, que permite adicionar URL's, efetuar pesquisas e consultar estatísticas.
2. **Gateway** - Componente central de coordenação, gere os barrels ativos, distribui pedidos e mantém estatísticas globais.
3. **Barrels** - Servidores responsáveis por armazenar o índice invertido das palavras (word → URL's), gerir stopwords e sincronizar dados entre si.
4. **Downloader** - Responsável por descarregar o conteúdo das páginas web, extrair palavras e ligações, e enviá-las para os Barrels.
5. **URL Queue** - Fila de processamento que gere a lista de URL's pendentes de download evita duplicação.

2.2 Componentes e Funções

2.2.1 Client

O **Client** é a aplicação final com a qual o utilizador interage.

Permite:

- Adicionar novos URL's ao sistema;
- Efetuar pesquisas por palavra ou conjunto de palavras;
- Consultar estatísticas globais (top 10 pesquisas, barrels ativos , tempos médios de resposta);
- Visualizar as ligações (outlinks) de uma determinada página.

O client comunica exclusivamente com o **Gateway**, através da interface remota **GatewayInterface**.

Desta forma, o utilizador não necessita conhecer a localização de outros módulos - pois é assegurada pelo Gateway.

2.2.2 Gateway

O **Gateway** é o núcleo de coordenação do sistema e atua como intermediário entre o cliente e os restantes componentes.

As suas principais funções são:

- **Balanceamento de carga (Load Balancing):** distribui as requisições de pesquisa entre os barrels de forma equitativa, utilizando um algoritmo **Round Robin**.
- **Gestão de falhas:** caso um barrel não responda, o Gateway tenta automaticamente o próximo barrel disponível (**failover**).
- **Registo e monitorização de barrels:** mantém uma lista de barrels ativos e registrados, atualizando dinamicamente a lista em caso de falhas ou reconexões.
- **Sincronização de dados:** quando um novo barrel é adicionado, o Gateway desencadeia um processo de **replicação e sincronização dos índices**, garantindo que o novo nó possui todos os dados já existentes nos outros barrels.
- **Gestão da fila de URLs:** estabelece ligação com o módulo **URLQueue**, que gerencia os URLs por processar.
- **Estatísticas globais:** mantém métricas como o top de pesquisas mais comuns e o tempo médio de resposta de cada barrel.

O Gateway comunica com:

- O **Client**, via **GatewayInterface**;
- Os **Barrels**, via **BarrellInterface**;
- A **URLQueue**, via **URLQueueInterface**.

2.2.3 Barrels

Os **Barrels** são os nós de armazenamento e indexação.

Cada barrel mantém:

- Um **índice invertido** (*word* → *set<URL>*);
- Um mapa de **ligações entre páginas** (*URL* → *set<associatedURLs>*);
- Estatísticas sobre **ligações de entrada** (*inbound links*);
- Um módulo de **gestão de stopwords** (*BarrelStopWords*), que identifica e ignora palavras muito frequentes ou pouco informativas.

Os barrels são executados de forma **independente** e registam-se dinamicamente no Gateway

A comunicação é feita através da interface **BarrelInterface**, que define operações remotas como:

- ***addToIndex(String word, String url)***
- ***searchWord(String word)***
- ***addUrlsForIndexedUrl(String url, HashSet<String> associatedUrls)***
- ***syncIndex(Map<String, HashSet<String>> otherIndex)***

Cada barrel também mantém persistência local dos seus dados através da classe **BarrelProgress**, que grava periodicamente o estado do índice em ficheiro. Assim, o sistema pode **recuperar automaticamente** o progresso após uma reinicialização.

A replicação é garantida pelo Gateway: quando um novo barrel é registado, o Gateway compara o seu índice com os já existentes e sincroniza as diferenças, assegurando **consistência entre réplicas**.

2.2.4 Downloader

O **Downloader** é o agente encarregado de **visitar páginas web e alimentar o sistema de indexação**.

As suas responsabilidades incluem:

- Obter o próximo URL da **URLQueue**;
- Fazer o download e parsing do conteúdo com **Jsoup**;
- Extrair texto e tokens (palavras);
- Contar a frequência das palavras (**wordCounts**);
- Extrair todas as ligações (outlinks) presentes na página;
- Distribuir as palavras e os links entre os **Barrels ativos**.
-

Durante este processo, o Downloader comunica com:

- O **Gateway**, para obter a lista atual de barrels disponíveis;
- A **URLQueue**, para receber e colocar URLs na fila;
- Os **Barrels**, para enviar dados de indexação (**addToIndex**, **addUrlsForIndexedUrl**, **addWordCounts**).

O componente é tolerante a falhas, tentando **até três vezes** processar cada URL antes de descartá-lo.

2.2.5 URLQueue

A **URLQueue** é o componente responsável pela **gestão concorrente de URLs pendentes**.

É implementada através de uma estrutura **thread-safe** (**LinkedBlockingDeque**) que permite múltiplos produtores e consumidores (Downloaders) trabalharem em simultâneo.

Funções principais:

- Armazenar URLs a serem processados, sem repetições (**seenUrls**);
- Permitir adição com prioridade (URLs inseridos pelo utilizador são processados primeiro);
- Fornecer URLs para download através de operações remotas como **takeNext()**.

Desta forma, a URLQueue garante **ordem, unicidade e sincronização** na distribuição de tarefas entre múltiplos downloaders.

2.3 Comunicação e Fluxo de Dados

O fluxo de dados principal do sistema pode ser descrito em quatro fases:

1. Submissão de URL (Client → Gateway → URLQueue)

O utilizador insere um URL no cliente, que é enviado ao Gateway.

O Gateway verifica se o URL já foi indexado e, caso contrário, adiciona-o à URLQueue.

2. Download e Extração (Downloader → URLQueue / Gateway → Barrels)

O Downloader obtém o próximo URL da fila, faz o download da página, processa o texto e envia as palavras e ligações para os barrels ativos.

3. Indexação (Barrels)

Cada barrel recebe as palavras, associa-as às URLs correspondentes e atualiza o índice invertido.

Palavras identificadas como stopwords são ignoradas.

4. Pesquisa (Client → Gateway → Barrels)

O cliente envia uma consulta ao Gateway, que a distribui entre os barrels. O Gateway agrega e ordena os resultados com base no número de referências (inbound links), retornando ao utilizador uma lista de URLs relevantes.

3. Funcionamento do RPC/RMI

A comunicação entre os componentes do sistema é implementada através de **RMI (Remote Method Invocation)**, um mecanismo de chamada remota de métodos em Java que permite que objetos localizados em diferentes máquinas ou processos se comuniquem de forma transparente, como se estivessem no mesmo espaço de memória.

Esta abordagem permite que o sistema funcione como uma **rede de serviços distribuídos**, onde cada módulo (Gateway, Barrel, Downloader, URLQueue e Client) expõe métodos remotos que podem ser invocados por outros módulos via rede.

3.1 Estrutura Geral de Comunicação RMI

Cada componente que precisa ser acessado remotamente implementa uma **interface remota** (que estende ***java.rmi.Remote***), define os métodos públicos disponíveis e cria uma **classe de implementação** que estende ***UnicastRemoteObject***.

Para cada componente, existe:

- Uma **interface remota**, que define o contrato (métodos acessíveis remotamente);
- Uma **classe de implementação**, que contém a lógica do serviço;
- Um **registro no RMI Registry**, onde o objeto remoto é associado a um nome para ser encontrado pelos outros módulos.

3.2 Interfaces Remotas

O projeto define quatro interfaces principais que estruturam toda a comunicação remota:

3.2.1 GatewayInterface

Esta interface define as operações expostas pelo **Gateway**, acessíveis tanto pelos **clientes** quanto pelos **downloaders e barrels**.

Principais métodos:

- ***searchWordGateway(String word)*** – Executa uma pesquisa simples por palavra.
- ***searchWords(List<String> words)*** – Pesquisa composta, podendo envolver interseção de resultados entre várias palavras.
- ***addURL(String url, boolean indexAnyway)*** – Adiciona um novo URL à fila para indexação.
- ***registerBarrel(String host, int port, String name)*** – Permite que barrels se registrem dinamicamente no Gateway.
- ***getActiveBarrels()*** e ***getRegisteredBarrels()*** – Fornecem listas de barrels ativos e registrados.
- ***getAverageResponseTime()*** – Devolve o tempo médio de resposta de cada barrel.
- ***getUrlsForIndexedUrl(String url)*** – Retorna as ligações (outlinks) de uma página indexada.

Esta interface é o ponto central de comunicação remota do sistema, conectando todos os módulos.

3.2.2 BarrelInterface

A interface que define as operações remotas dos **IndexStorageBarrels**. É utilizada pelo **Gateway** e pelo **Downloader**.

Métodos principais:

- ***addToIndex(String word, String url)*** – Adiciona uma palavra e o URL correspondente ao índice.
- ***searchWord(String word)*** – Retorna as URLs associadas a uma palavra.

- ***addUrlsForIndexedUrl(String url, HashSet<String> associatedUrls)*** – Regista as ligações encontradas numa página.
- ***addWordCounts(Map<String, Integer> wordCounts, String url)*** – Envia a contagem de palavras de uma página para cálculo de stopwords.
- ***syncIndex(Map<String, HashSet<String>> otherIndex)*** – Sincroniza índices entre barreils (replicação).
- ***getIndex()*** – Retorna o índice completo armazenado no barrel.
- ***getInboundLinkCounts()*** – Retorna o número de ligações recebidas (inbound links) por URL.
- ***hasIndexedUrl(String url)*** – Verifica se um URL já foi processado.

Esta interface é essencial para a cooperação entre os diferentes barreils e para o processo de replicação controlado pelo Gateway.

3.2.3 URLQueueInterface

A interface que define os métodos remotos de interação com a **fila de URLs**.

É usada tanto pelo **Gateway** (para adicionar novos URLs) quanto pelos **Downloaders** (para consumir URLs).

Métodos principais:

- ***takeNext()*** – Retira e devolve o próximo URL a processar.
- ***putNew(String url, boolean priority)*** – Adiciona um novo URL à fila (com prioridade se o pedido vier do utilizador).
- ***getQueueSize()*** – Retorna o tamanho atual da fila.

Esta interface implementa o padrão produtor-consumidor distribuído, permitindo o balanceamento natural entre múltiplos downloaders.

3.2.4 DownloaderInterface

Interface mais simples do sistema, usada principalmente para monitorização.

Define apenas um método:

- **getProcessorURLsCount()** – Retorna o número total de URLs processados pelo downloader.

4. Replicação e Reliable Multicast

A arquitetura do sistema inclui um mecanismo de **replicação de dados entre barrels** (os servidores de índices), garantindo **disponibilidade, consistência parcial e tolerância a falhas**.

O objetivo é que, mesmo que um barrel falhe, o sistema continue a responder a pesquisas e a manter o índice global atualizado.

A replicação é feita de forma **controlada pelo Gateway**, que atua como coordenador da sincronização entre as instâncias dos barrels.

4.1 Reliable Multicast

Embora não utilize um protocolo de multicast real, o sistema implementa uma **propagação confiável de estado (reliable multicast)** a nível lógico.

Cada atualização relevante (novos URLs, novas palavras, links ou contagens de palavras) é replicada através de invocações RMI para **todos os barrels ativos**, garantindo que:

- Todos os nós recebem as atualizações;
- O sistema permanece consistente, mesmo que um barrel falhe temporariamente.

Exemplo:

Quando o **Downloader** processa uma página e extrai as palavras:

```
barrelInterface.addToIndex(novaPalavra, url);
```

Este método é chamado para todos os barrels ativos retornados por:

```
List<String> activeBarrels = gateway.getActiveBarrels();
```

Assim, cada barrel mantém uma cópia sincronizada do índice global, recebendo o mesmo conjunto de dados de indexação.

5. Distribuição de Tarefas

O desenvolvimento deste projeto foi realizado de forma colaborativa e equilibrada, com todos os elementos do grupo a participarem de maneira ativa em todas as fases do processo.